Azure Kubernetes Service (AKS) Secure Baseline

Implementation Solution Guide

Version 1.0.0 · 2026-07-10

Contents

Executive Summary — Azure Kubernetes Service (AKS) Secure Baseline (Microsoft Azure) 4

What you get . . . 4

What this delivers . . . 4

The standard deployment model . . . 4

Outcome . . . 4

1. Solution Overview — Azure Kubernetes Service (AKS) Secure Baseline 5

What the AKS secure baseline is . . . 5

Why this matters . . . 5

How it works . . . 5

What you can control . . . 6

Limitations (be honest with buyers) . . . 6

Auth and connections used . . . 7

The standard deployment model . . . 7

2. Requirements Guide — Azure Kubernetes Service (AKS) Secure Baseline 7

2.1 An Azure subscription with sufficient rights . . . 7

2.2 A virtual network and subnet for the cluster . . . 7

2.3 Entra ID groups for cluster access (least privilege) . . . 8

2.4 IaC tooling and Kubernetes CLI on the operator workstation . . . 8

2.5 Azure Policy for Kubernetes add-on available . . . 8

2.6 Workload identity / OIDC prerequisites . . . 8

2.7 Microsoft Defender for Containers plan decision . . . 9

2.8 Change control, non-prod cluster, and rollback . . . 9

Checklist . . . 9

Global Requirements — Azure Solutions 9

G1. A Microsoft 365 / Entra tenant . . . 9

G2. Administrative roles (least privilege) . . . 10

G3. Licensing baseline . . . 10

G4. Network connectivity to Microsoft 365 / Azure endpoints . . . 10

G5. Change control & maintenance window . . . 11

G6. Backup / rollback posture . . . 11

3. Implementation Guide — Azure Kubernetes Service (AKS) Secure Baseline 11

Naming (set once, reuse verbatim) . . . 11

Step 1 — Assess and set variables . . . 12

Step 2 — Plan the IaC (the gate) . . . 12

Step 3 — Deploy the private non-prod cluster . . . 12

Step 4 — Connect over the private endpoint . . . 12
Step 5 — Apply Kubernetes RBAC (least privilege) . . . 13
Step 6 — Assign Azure Policy for Kubernetes in audit . . . 13
Step 7 — Wire workload identity (secretless pod auth) . . . 13
Step 8 — Enable Microsoft Defender for Containers . . . 13
Step 9 — Validate on the non-prod cluster . . . 14
Step 10 — Move policy from audit to deny (the gate) . . . 14
Step 11 — Expand to production . . . 14
5. Testing Guide — Azure Kubernetes Service (AKS) Secure Baseline 15
Test 1 — API server is private . . . 15
Test 2 — Entra + Azure RBAC for Kubernetes . . . 15
Test 3 — Least-privilege RBAC bindings present . . . 15
Test 4 — Azure Policy add-on / Gatekeeper healthy . . . 15
Test 5 — Audit reports, does not block (audit stage) . . . 15
Test 6 — Workload identity is secretless . . . 15
Test 7 — Defender for Containers scanning . . . 16
Test 8 — Deny blocks (after the gate) . . . 16
Test 9 — Posture report . . . 16
Pass/fail gate . . . 16
6. Troubleshooting Guide — Azure Kubernetes Service (AKS) Secure Baseline 16
A. `terraform apply` / IaC failures . . . 16
B. `kubectl` can't reach the cluster (private API) . . . 17
C. Access denied for the right people . . . 17
D. Azure Policy for Kubernetes / Gatekeeper . . . 18
E. Workload identity not working (pod can't reach Azure) . . . 18
F. Defender for Containers has no findings . . . 19
G. Diagnostics quick reference . . . 19
H. When to escalate . . . 19
7. Best Practices — Azure Kubernetes Service (AKS) Secure Baseline 19
Where to go next . . . 20
8. Deliverables Summary — Azure Kubernetes Service (AKS) Secure Baseline 20
Documentation . . . 20
Infrastructure-as-Code & scripts . . . 21
What the buyer receives . . . 21
Outcome . . . 22
Appendix A — PowerShell Scripts 23

Executive Summary — Azure Kubernetes Service (AKS) Secure Baseline (Microsoft Azure)

White-label · Azure Solutions

A default AKS cluster is a soft target: a **public API server** reachable from the internet, **broad Kubernetes RBAC**, **no admission policy**, and application secrets sitting in environment variables. This solution replaces that default with an **AKS secure baseline** deployed **as code** — a private cluster, Entra ID identity mapped to least-privilege Kubernetes RBAC, Azure Policy for Kubernetes (Gatekeeper) guardrails, workload identity for secretless pod-to-Azure auth, and Microsoft Defender for Containers with image scanning and node hardening.

This is a complete, **deploy-it-today** package for standing up a production-grade AKS baseline, written for an administrator with no prior Terraform, Bicep, or Kubernetes-security experience. Every deploy is **plan / what-if first** and proven on a **non-production** cluster before it touches anything real.

What you get

- A plain-language **Solution Overview** of private cluster networking, Entra ID + Kubernetes RBAC, Azure Policy (Gatekeeper), workload identity, and Defender for Containers.
- A **Requirements Guide** with verify / implement-if-missing for every prerequisite.
- An **Implementation Guide** of precise, ordered steps — Terraform/Bicep plan → deploy the private cluster → apply Kubernetes RBAC and policy manifests → wire workload identity → enable Defender — non-prod first.
- Real **Infrastructure-as-Code**: a Terraform configuration for the cluster, kubectl **YAML** manifests for RBAC / policy / workload identity, and three **Az PowerShell** helpers for discovery, validation, and posture reporting.
- **Testing** and **Troubleshooting** guides for confident rollout.

What this delivers

A **private AKS cluster** (private API server, Azure CNI, internal ingress) defined in code, **Entra ID authentication** with **Azure RBAC for Kubernetes** granting least-privilege access by group, **Azure Policy for Kubernetes** guardrails enforced through Gatekeeper (baseline/restricted Pod Security, no privileged containers, approved registries only), **workload identity** so pods authenticate to Azure with a federated token instead of a stored secret, and **Microsoft Defender for Containers** scanning images and hardening nodes — all validated on a non-production cluster first.

The standard deployment model

Assess → **Configure** → **Pilot** → **Validate** → **Expand** — with a **plan / what-if gate** before every apply and a documented rollback (`terraform destroy` of the non-prod stack / policy set to audit).

Outcome

Kubernetes running securely on Azure — a private, policy-guardrailed cluster with least-privilege identity and secretless pod auth — instead of an internet-exposed default cluster, deployable by a single administrator and validated before you scale to production.

1. Solution Overview — Azure Kubernetes Service (AKS) Secure Baseline

What the AKS secure baseline is

The AKS secure baseline is a **hardened, production-grade way to run Kubernetes on Azure**, expressed as infrastructure-as-code so it is repeatable and auditable rather than hand-clicked. It combines five capabilities into one cluster:

- **Private cluster networking** — the Kubernetes API server has a **private endpoint** (no public IP), the cluster uses **Azure CNI** networking inside your virtual network, and application traffic enters through an **internal ingress** rather than a public load balancer.
- **Entra ID + Kubernetes RBAC** — cluster sign-in uses **Microsoft Entra ID**, and access is authorized with **Azure RBAC for Kubernetes**, so an Entra group maps to a least-privilege Kubernetes role instead of everyone sharing the cluster-admin credential.
- **Azure Policy for Kubernetes (Gatekeeper)** — the Azure Policy add-on installs the **Gatekeeper** admission controller, which enforces guardrails at deploy time (no privileged pods, approved registries only, Pod Security baseline/restricted, resource limits required).
- **Workload identity (federated)** — pods authenticate to Azure services with a **federated Entra token** exchanged for a managed identity, so there are **no secrets or connection strings** stored in the pod.
- **Microsoft Defender for Containers** — continuous **image scanning**, runtime threat detection, and **node hardening** recommendations across the cluster.

Why this matters

A cluster created with defaults is exposed on several fronts at once. The API server answers on a public IP, so it is discoverable and brute-forceable. Kubernetes RBAC is broad — a single kubeconfig with cluster-admin gets shared around. There is no admission policy, so anyone who can deploy can run a privileged container, pull an image from any registry, or mount the host filesystem. And applications hold Azure secrets in environment variables, which leak through logs, crash dumps, and `kubectl describe`. The secure baseline closes all of these by construction: the API is private, identity is Entra-backed and least-privilege, policy blocks the dangerous deployment at admission, and pods carry no secret to leak. Because the whole cluster is defined as code, you prove it once on a non-production cluster, then apply the same definition to production with a plan/what-if gate.

How it works

1. An **Infrastructure-as-Code plan** (Terraform `plan` or Bicep `what-if`) describes the cluster and is reviewed **before** anything is created — no blind apply.

2. The IaC deploys a **private AKS cluster** into an existing virtual network subnet: private API endpoint, Azure CNI, a system node pool with host-based OS, Entra integration, Azure RBAC for Kubernetes, the Azure Policy add-on, OIDC issuer, and workload-identity enabled.

3. **Kubernetes RBAC** is applied from **YAML manifests** — namespaces, and `RoleBinding`/`ClusterRoleBinding` objects that map **Entra group object IDs** to least-privilege roles (namespace developer, cluster viewer) instead of cluster-admin.

4. **Azure Policy for Kubernetes** initiatives are assigned to the cluster in **audit** first (Gatekeeper reports violations without blocking), then moved to **deny** once the picture is clean.

5. **Workload identity** is wired: a **user-assigned managed identity** is created, a **federated credential** ties it to a Kubernetes **service account**, and pods that use that service account get an Azure token with **no stored secret**.

6. **Microsoft Defender for Containers** is enabled on the subscription so images are scanned and nodes are assessed for hardening, and findings feed the Defender for Cloud secure score.

What you can control

• **Network exposure** — private vs public API, authorized IP ranges (if a public endpoint is unavoidable), Azure CNI address space, and internal vs public ingress.

• **Identity and access** — which Entra groups map to which Kubernetes roles, and whether local (cluster-admin) accounts are disabled.

• **Admission guardrails** — which Azure Policy for Kubernetes initiatives apply, and whether each is in **audit** or **deny**.

• **Workload identity** — which managed identity a service account federates to, and which Azure resources (Key Vault, Storage, SQL) that identity may reach.

• **Threat protection** — Defender for Containers coverage and which image-scan / node-hardening findings you remediate first.

Limitations (be honest with buyers)

• A **private cluster** means the API server is only reachable from inside the virtual network (or a peered network / VPN / Bastion jumpbox). You must plan management connectivity — CI/CD runners and admin workstations need a network path to the private endpoint.

• **Azure Policy for Kubernetes** and **workload identity** availability, add-on versions, and some Pod Security features move between GA and preview — *verify current availability and API/add-on versions in your tenant and region before you rely on them.*

• **Microsoft Defender for Containers** is a **paid plan** billed per vCore/resource; the free posture view is limited. *Verify current pricing in your subscription.*

• A **deny** policy blocks *new* non-compliant workloads at admission; it does not reconfigure workloads already running — those are remediated separately.

• The baseline hardens the platform. It does not replace application security, image supply-chain controls upstream of the cluster, patching cadence, or backup of stateful workloads.

Auth and connections used

• **Microsoft Entra ID** — cluster authentication (Entra-integrated AKS) and the identity behind Azure RBAC for Kubernetes and workload identity.

• **Azure Resource Manager** — the Terraform/Bicep provider and Az PowerShell operate against ARM with **Owner** or scoped **Contributor + RBAC Administrator** on the target resource group/subscription.

• **Kubernetes API (private endpoint)** — `kubectl` applies the RBAC and policy manifests over the private endpoint using an Entra token.

• **OIDC federation** — the cluster's OIDC issuer plus an Entra federated credential back workload identity token exchange.

The standard deployment model

> ***Assess prerequisites → plan the IaC (terraform plan / bicep what-if) → deploy the private cluster (non-prod) → apply Kubernetes RBAC + policy (audit) + workload identity → enable Defender for Containers → validate on the non-prod cluster → move policy to deny → expand to production.*** *repeatable pattern:* ***Assess → Configure → Pilot → Validate → Expand****, with a* ***plan/what-if gate*** *before every apply and a documented rollback.*

2. Requirements Guide — Azure Kubernetes Service (AKS) Secure Baseline

Verify **every** requirement before deploying. Each entry: **Description · Why · Verify · Implement-if-missing.** Category-wide prerequisites are in `global-requirements-azure-solutions.md` (tenant, admin roles, licensing, connectivity, change control, rollback) and are not repeated here. Objects are named `<Purpose> – <Scope> – <Stage>` and reused verbatim.

2.1 An Azure subscription with sufficient rights

Description: A subscription under your Entra tenant, and an operating account with **Owner** (or **Contributor** + **Role Based Access Control Administrator**) on the target resource group / subscription. **Why:** Creating the cluster, assigning Azure RBAC for Kubernetes, and creating role assignments for the managed identity all require rights to write resources *and* role assignments. **Verify:** `portal.azure.com` → **Subscriptions** → **Access control (IAM)**; run `AKSSEC-Discovery.ps1`. **Implement-if-missing:** Assign the minimum roles at the narrowest scope; prefer **PIM eligible** activation over standing access.

2.2 A virtual network and subnet for the cluster

Description: An existing **virtual network** with a dedicated **subnet** sized for Azure CNI (one IP per pod plus nodes), and a plan for management connectivity to the **private** API endpoint. **Why:** A private, CNI-networked cluster lives inside your VNet; the subnet must have enough address space, and admins/CI need a network path (peering, VPN, Bastion, or a private DNS-resolving jumpbox) to reach the private API. **Verify:** `portal.azure.com` → **Virtual networks** → confirm the subnet and free address

count; AKSSEC-Discovery.ps1 reports VNet/subnet capacity. **Implement-if-missing:** Create/resize a subnet with headroom (size for max pods × max nodes); stand up a peered management network, VPN, or a jumpbox VM that can resolve the private endpoint.

2.3 Entra ID groups for cluster access (least privilege)

Description: Entra **security groups** that map to Kubernetes roles — at minimum a **cluster admin** group and a **namespace developer** group; capture their **object IDs**. **Why:** Azure RBAC for Kubernetes and the RBAC manifests bind **Entra groups**, not individuals, to least-privilege roles — this is what replaces a shared cluster-admin kubeconfig. **Verify:** `entra.microsoft.com` → **Groups** → confirm the groups and copy each **Object ID**; AKSSEC-Discovery.ps1 echoes the IDs you supply. **Implement-if-missing:** Create the groups (e.g. `AKS Admins – Baseline – Prod`, `AKS Developers – Baseline – Prod`) and add members; record the object IDs for the RBAC manifests.

2.4 IaC tooling and Kubernetes CLI on the operator workstation

Description: Terraform (or the **Bicep**/Azure CLI toolchain), **Az PowerShell** (Az.Accounts, Az.Aks, Az.Resources, Az.Security), **Azure CLI** with the `aks-preview` extension where a feature requires it, and **kubectl** + **kubelogin**. **Why:** The cluster is deployed by IaC and the RBAC/policy/workload-identity objects are applied with `kubectl`; `kubelogin` is required for Entra-integrated cluster sign-in. **Verify:** `terraform version`, `az version`, `kubectl version --client`, `kubelogin --version`; AKSSEC-Validate.ps1 checks tool presence. **Implement-if-missing:** Install Terraform / Azure CLI / Az PowerShell / kubectl / kubelogin; register the `aks-preview` extension only if a chosen feature is still preview.

2.5 Azure Policy for Kubernetes add-on available

Description: The **Azure Policy add-on** for AKS (Gatekeeper) and the **Microsoft.PolicyInsights** / **Microsoft.ContainerService** resource providers registered on the subscription. **Why:** The add-on installs Gatekeeper and lets Azure Policy initiatives enforce admission guardrails; the providers must be registered before the add-on installs. **Verify:** `az provider show -n Microsoft.PolicyInsights` and `Microsoft.ContainerService` show **Registered**; AKSSEC-Validate.ps1 checks provider + add-on state. **Implement-if-missing:** `az provider register --namespace Microsoft.PolicyInsights` (and `Microsoft.ContainerService`); the add-on is enabled by the IaC (`azure_policy_enabled = true`). *Verify current add-on availability in your region.*

2.6 Workload identity / OIDC prerequisites

Description: The cluster's **OIDC issuer** and **workload identity** enabled, plus a **user-assigned managed identity** the pods will federate to, and a decision on which Azure resources that identity may reach. **Why:** Workload identity exchanges the cluster's OIDC token for an Entra token tied to the managed identity — this is what makes pod-to-Azure auth **secretless**. **Verify:** After deploy, `az aks show ... --query oidcIssuerProfile.enabled` and `securityProfile.workloadIdentity.enabled` return `true`; AKSSEC-Validate.ps1 confirms both. **Implement-if-missing:** The IaC sets `oidc_issuer_enabled = true` and `workload_identity_enabled = true` and creates the managed identity; grant that identity least-privilege access to the target resource (e.g. Key Vault secrets user). *Verify current workload-identity GA status in your region.*

2.7 Microsoft Defender for Containers plan decision

Description: A decision to enable **Microsoft Defender for Containers** on the subscription (image scanning, runtime detection, node hardening). **Why:** Image-scan findings, runtime alerts, and node-hardening recommendations come from this Defender plan; it is **paid** per resource. **Verify:** `portal.azure.com` → **Microsoft Defender for Cloud** → **Environment settings** → **Containers** plan status; `AKSSEC-Discovery.ps1` reports the plan tier. **Implement-if-missing:** Enable the Containers plan on the in-scope subscription. *Verify current pricing before enabling.*

2.8 Change control, non-prod cluster, and rollback

Description: An approved change window, a named **non-production** cluster to prove the baseline on first, and a documented rollback before any production apply or policy move to deny. **Why:** IaC applies and deny policies can disrupt real workloads; the plan/what-if gate, a non-prod-first proof, and a rollback are the safety net (per `global-requirements-azure-solutions.md` G5/G6). **Verify:** An approved change record, the non-prod cluster name, and a written back-out exist. **Implement-if-missing:** Define the non-prod cluster (`AKS Cluster – Baseline – NonProd`), capture the current state (`AKSSEC-Report.ps1`), and write the rollback (`terraform destroy` of the non-prod stack / set policy assignments to **audit**).

Checklist

- Subscription + Owner (or Contributor + RBAC Administrator) — `AKSSEC-Discovery.ps1`
- VNet + CNI-sized subnet + management path to the private API
- Entra admin & developer groups (object IDs captured)
- Terraform / Az PowerShell / Azure CLI / kubectl / kubelogin installed — `AKSSEC-Validate.ps1`
- Microsoft.PolicyInsights + Microsoft.ContainerService registered; Azure Policy add-on available
- OIDC issuer + workload identity + user-assigned managed identity plan
- Defender for Containers plan decision made
- Change window, non-prod cluster, and rollback documented

Global Requirements — Azure Solutions

These requirements are common to **every** Azure Solutions solution in the standard program model. They are the baseline platform prerequisites that any Azure/Microsoft 365 engagement assumes. Each is structured **Description / Why / Verify / Implement-if-missing**. Solution-specific requirements (see `02-requirements.md`) are layered on top of these.

> *Naming convention: every created object is named `<Purpose> – <Scope> – <Stage>` and reused verbatim throughout the engagement.*

G1. A Microsoft 365 / Entra tenant

• **Description:** An active Microsoft 365 / Microsoft Entra tenant for the cloud(s) in scope (commercial and/or government).

• **Why:** All identity, licensing, and service configuration is anchored to a tenant; nothing can be configured without one.

• **Verify:** Sign in to the relevant admin center (`https://admin.microsoft.com`) and confirm the tenant resolves and is active under **Settings** → **Org settings**.

• **Implement-if-missing:** Provision the tenant through the appropriate channel (commercial: standard; government: via an authorized AOS-G reseller).

G2. Administrative roles (least privilege)

• **Description:** Appropriately scoped admin roles for the work — Global Administrator for tenant-wide changes, and service-specific roles (Exchange, SharePoint, Teams, Azure Owner/Contributor) for day-to-day.

• **Why:** Every configuration and migration action requires a role with sufficient privilege; least-privilege scoping limits blast radius and satisfies audit requirements.

• **Verify:** In Entra → **Roles & admins**, confirm the operating accounts hold the required roles. In Azure → **Subscriptions** → **Access control (IAM)**, confirm RBAC assignments.

• **Implement-if-missing:** Assign the minimum roles needed; prefer eligible (PIM) activation over standing access where available.

G3. Licensing baseline

• **Description:** Sufficient Microsoft 365 and/or Azure licensing/credit for all in-scope users and workloads.

• **Why:** Features and capacity (mailboxes, SharePoint storage, AVD, VMs) are gated by license SKUs and subscription quota.

• **Verify: Billing** → **Licenses** for M365; **Subscriptions** → **Usage + quotas** for Azure. Compare assigned/available against the in-scope object count.

• **Implement-if-missing:** Procure the correct SKUs and request quota increases sized to the workload (add headroom for shared/resource objects).

G4. Network connectivity to Microsoft 365 / Azure endpoints

• **Description:** Reliable network egress to the required Microsoft 365 and Azure service endpoints (and the sovereign `.us` endpoints for government clouds), with sufficient bandwidth.

• **Why:** All cloud administration and any data movement depends on connectivity to the correct endpoint set; government clouds use distinct endpoints from commercial.

• **Verify:** Confirm reachability to the relevant endpoints (commercial `*.microsoft.com` / government `*.microsoft.us`, `*.azure.us`, `*.office365.us`). Measure sustained bandwidth against the data volume.

• **Implement-if-missing:** Open firewall/proxy allowances for the correct endpoint set; scale bandwidth or add ExpressRoute (matching the cloud — Gov ExpressRoute for Gov regions).

G5. Change control & maintenance window

• **Description:** An approved change-management process with scheduled maintenance windows and a stakeholder communications plan.

• **Why:** Cloud changes can be outward-facing and disruptive; controlled windows and comms reduce business impact and satisfy governance/compliance.

• **Verify:** Confirm an approved change record and a scheduled window for each disruptive action.

• **Implement-if-missing:** Stand up a lightweight change/approval process; schedule windows; prepare user comms templates.

G6. Backup / rollback posture

• **Description:** A defined backup and rollback position for any change — config export, data backup, and a documented back-out procedure before any disruptive action.

• **Why:** Cloud changes are not always reversible by default; a tested rollback is the safety net for cutovers and bulk operations.

• **Verify:** Confirm pre-change config/state is captured and a written rollback exists for each disruptive change.

• **Implement-if-missing:** Export current configuration/state; document and (where feasible) rehearse the rollback before executing the change.

3. Implementation Guide — Azure Kubernetes Service (AKS) Secure Baseline

Follow these steps **in order**. The cluster is deployed by **Infrastructure-as-Code** (Terraform in this guide — the Bicep `what-if` flow is equivalent), then RBAC, policy, and workload identity are applied with `kubectl`. Objects are named `<Purpose> – <Scope> – <Stage>` and reused verbatim in every later step, script, and test.

> ***Model:*** *Everything runs **plan / what-if first** and on the **non-production** cluster first. Assign Azure Policy for Kubernetes in **audit**, prove the baseline, then move the **same** assignment to **deny** on an approved gate. Keep a rollback (`terraform destroy` of the non-prod stack / policy back to audit) for every change.*

Naming (set once, reuse verbatim)

• Non-prod cluster: `AKS Cluster – Baseline – NonProd`

• Production cluster: `AKS Cluster – Baseline – Prod`

• Resource group: `AKS RG – Baseline – NonProd`

• Entra admin group: `AKS Admins – Baseline – Prod`

- Entra developer group: `AKS Developers – Baseline – Prod`
- Application namespace: `app-baseline`
- Workload identity managed identity: `AKS Workload Identity – Baseline – NonProd`
- Kubernetes service account (federated): `wi-app-baseline`
- Policy assignment (audit stage): `AKS Guardrails – NonProd – Audit`
- Policy assignment (deny stage): `AKS Guardrails – NonProd – Deny`

Step 1 — Assess and set variables

1. Run `AKSSEC-Discovery.ps1` to capture the subscription, VNet/subnet capacity, existing clusters, and Defender plan — this is your pre-change baseline.

2. Open the Terraform variables file (`AKSSEC-main.tf` header / a `terraform.tfvars` you create) and set: resource group `AKS RG – Baseline – NonProd`, region, existing **VNet/subnet IDs**, the **Entra admin group object ID** (`AKS Admins – Baseline – Prod`), and node sizing.

3. Confirm the operator workstation has a **network path to the private API endpoint** (peering / VPN / jumpbox) — the later `kubectl` steps depend on it.

Step 2 — Plan the IaC (the gate)

1. `az login`, then `terraform init` in the script folder.

2. Run `terraform plan -out aksbaseline.plan`. **Review every resource** it will create: the private AKS cluster, the OIDC issuer, workload identity, the Azure Policy add-on, and the role assignments. **Nothing is created yet.**

3. (Bicep equivalent: `az deployment group what-if` — review the same resource set.)

4. Only continue when the plan matches intent and the change is approved.

Step 3 — Deploy the private non-prod cluster

1. Run `terraform apply aksbaseline.plan`. This creates `AKS Cluster – Baseline – NonProd` with: **private API endpoint** (`private_cluster_enabled = true`), **Azure CNI**, **Entra integration + Azure RBAC for Kubernetes** (`azure_rbac_enabled = true`, local accounts disabled), **Azure Policy add-on** (`azure_policy_enabled = true`), **OIDC issuer** and **workload identity** enabled.

2. When apply completes, confirm outputs (cluster name, OIDC issuer URL, node resource group).

3. Run `AKSSEC-Validate.ps1` — it confirms private API, Azure RBAC, the policy add-on, OIDC, and workload identity are all on.

Step 4 — Connect over the private endpoint

1. `az aks get-credentials --resource-group "<AKS RG – Baseline – NonProd>" --name "<AKS Cluster – Baseline – NonProd>"`.

2. Because Entra + Azure RBAC is enabled, `kubelogin convert-kubeconfig -l azurecli` prepares the kubeconfig for Entra sign-in.

3. From a workstation **on the cluster's network**, run `kubectl get nodes` — it should list the node pool. (If it times out, you are not on a network that can reach the private endpoint — see Troubleshooting.)

Step 5 — Apply Kubernetes RBAC (least privilege)

1. Edit `AKSSEC-baseline.yaml` and replace the placeholder Entra **group object IDs** with `AKS Admins – Baseline – Prod` and `AKS Developers – Baseline – Prod`.

2. `kubectl apply -f AKSSEC-baseline.yaml` creates the `app-baseline` namespace, a `ClusterRoleBinding` mapping the admin group to a scoped admin role, and a namespaced `RoleBinding` mapping the developer group to an **edit** role in `app-baseline` only — **not** cluster-admin.

3. Confirm no user holds the local cluster-admin credential (local accounts were disabled in Step 3).

Step 6 — Assign Azure Policy for Kubernetes in audit

1. `portal.azure.com` → **Policy** → **Definitions** → filter category **Kubernetes**. Select the built-in **initiative** for cluster pod security baseline/restricted plus the individual guardrails you need (no privileged containers, allowed images/registries only, resource limits required, no host path/namespace).

2. **Assign** at the **cluster's resource-group scope**. **Assignment name:** `AKS Guardrails – NonProd – Audit`. Set every effect to **audit**.

3. Gatekeeper reports violations without blocking. Give it time to sync, then review **Policy** → **Compliance**.

4. Run `AKSSEC-Report.ps1` to capture the audit-stage compliance picture.

Step 7 — Wire workload identity (secretless pod auth)

1. Confirm the IaC created the user-assigned managed identity `AKS Workload Identity – Baseline – NonProd` and its **federated credential** pointing at the cluster's OIDC issuer + the service account `system:serviceaccount:app-baseline:wi-app-baseline`.

2. In `AKSSEC-baseline.yaml`, the `ServiceAccount` `wi-app-baseline` carries the annotation `azure.workload.identity/client-id: <managed identity client id>` and the sample pod carries the label `azure.workload.identity/use: "true"`. Set the client ID and `kubectl apply -f AKSSEC-baseline.yaml`.

3. Grant the managed identity **least-privilege** access to the target Azure resource (e.g. **Key Vault Secrets User** on one vault) — do **not** grant broad rights.

4. Deploy the sample workload; it should read the Azure resource using the **federated token** with **no secret** in the pod spec.

Step 8 — Enable Microsoft Defender for Containers

1. `portal.azure.com` → **Microsoft Defender for Cloud** → **Environment settings** → the subscription → enable the **Containers** plan (agreed in requirements).

2. Confirm the Defender extension/agent deploys to the cluster (it installs automatically for the plan).

3. **Defender for Cloud** → **Recommendations** — review **image-scan** findings for your registry and **node-hardening** recommendations; remediate the highest-severity first.

4. Run `AKSSEC-Report.ps1` to capture Defender coverage and top findings alongside the policy compliance picture.

Step 9 — Validate on the non-prod cluster

1. Run `AKSSEC-Validate.ps1` → private API, Azure RBAC, policy add-on, OIDC, workload identity all confirmed.

2. Complete the **Testing Guide** — the deny-blocks-privileged test in **audit** should *report* but not block yet.

3. Only proceed to the gate when every non-prod test passes.

Step 10 — Move policy from audit to deny (the gate)

1. **Gate:** proceed only when the audit picture is clean (violations understood, legitimate exceptions exempted) and the change is approved.

2. **Policy** → **Assignments** → duplicate `AKS Guardrails – NonProd – Audit` as `AKS Guardrails – NonProd – Deny`, or edit it, setting each guardrail **effect** to **deny**.

3. Re-run the Testing Guide privileged-pod test — Gatekeeper must now **reject** the deployment at admission; a compliant workload still deploys.

4. **Rollback:** if deny blocks legitimate work, set the effect back to **audit** (or remove the deny assignment) — running workloads are unaffected.

Step 11 — Expand to production

1. Re-run the IaC targeting the production resource group/VNet to create `AKS Cluster – Baseline – Prod` (same definition, prod variables) — **plan first**.

2. Apply the RBAC manifests with the same Entra groups, and assign the **validated** policy initiative (in the effect stage you proved) at the prod cluster scope.

3. Wire workload identity for prod (a prod managed identity + federated credential) and enable Defender for Containers on the prod scope.

4. Re-validate each cluster with `AKSSEC-Validate.ps1` and track posture/drift with `AKSSEC-Report.ps1`.

Order of operations

`Assess → terraform plan (gate) → deploy private non-prod cluster → connect over private endpoint → apply Kubernetes RBAC → assign policy (audit) → wire workload identity → enable Defender → validate → move policy to deny (gate)`

→ expand to prod

5. Testing Guide — Azure Kubernetes Service (AKS) Secure Baseline

Run after the non-prod deploy (Implementation Step 9), again after moving policy to deny (Step 10), and after each expansion. Objects are referenced by their <Purpose> – <Scope> – <Stage> names, and the scripts are AKSSEC-Discovery.ps1, AKSSEC-Validate.ps1, AKSSEC-Report.ps1.

Test 1 — API server is private

Do: az aks show -g "<AKS RG – Baseline – NonProd>" -n "<AKS Cluster – Baseline – NonProd>" --query apiServerAccessProfile.enablePrivateCluster; or run AKSSEC-Validate.ps1. **Expected:** Returns **true**. The API server has a private endpoint; a kubectl call from *off* the cluster network times out (this is the point).

Test 2 — Entra + Azure RBAC for Kubernetes

Do: From a workstation on the cluster network, kubectl get nodes as a member of AKS Admins – Baseline – Prod; then as a member of AKS Developers – Baseline – Prod. **Expected:** The admin sign-in triggers Entra auth and lists nodes; the developer can act **only** inside app-baseline and is denied cluster-wide admin. No one uses a local cluster-admin credential (local accounts disabled).

Test 3 — Least-privilege RBAC bindings present

Do: kubectl get clusterrolebinding,rolebinding -A | grep baseline; or run AKSSEC-Validate.ps1. **Expected:** The admin ClusterRoleBinding and the app-baseline developer RoleBinding exist and reference the Entra **group object IDs** — not individual users, not cluster-admin for developers.

Test 4 — Azure Policy add-on / Gatekeeper healthy

Do: kubectl get pods -n gatekeeper-system; **Policy** → **Compliance** for AKS Guardrails – NonProd – Audit; or run AKSSEC-Validate.ps1. **Expected:** Gatekeeper pods are running and the audit-stage assignment shows a per-constraint compliant/non-compliant breakdown.

Test 5 — Audit reports, does not block (audit stage)

Do: Deploy a deliberately non-compliant pod (privileged, or an image from a disallowed registry) into app-baseline. **Expected:** The pod **is created** but shows as **non-compliant** in Policy compliance — audit reports, it does not block.

Test 6 — Workload identity is secretless

Do: Deploy the sample workload using service account wi-app-baseline (label azure.workload.identity/use: "true") and have it read the granted Azure resource (e.g. a Key Vault secret). **Expected:** The workload authenticates with a **federated token** and reads the resource with

no secret / connection string in the pod spec. `kubectl describe pod` shows no injected secret credential.

Test 7 — Defender for Containers scanning

Do: Microsoft Defender for Cloud → **Recommendations**; or run `AKSSEC-Report.ps1`. **Expected:** The Containers plan is enabled, images show scan results, and node-hardening recommendations are listed for the cluster.

Test 8 — Deny blocks (after the gate)

Do: After Step 10, re-deploy the privileged / disallowed-registry pod from Test 5. **Expected:** Gatekeeper **rejects** the deployment at admission with a policy message; a compliant pod still deploys successfully.

Test 9 — Posture report

Do: Run `AKSSEC-Report.ps1`. **Expected:** A clear baseline report — private-API state, Azure RBAC, RBAC bindings, policy assignments and compliance, workload-identity/OIDC state, and Defender coverage with top findings.

Pass/fail gate

Gate	Requirement
Non-prod pass (pre-deny)	Tests 1–7 pass; the audit picture is understood and exceptions exempted
Deny pass	Test 8 passes — a privileged/disallowed pod is blocked and a compliant one still deploys
Expand pass	Tests 1–4, 6, 9 pass on the production cluster before it carries real workloads

6. Troubleshooting Guide — Azure Kubernetes Service (AKS) Secure Baseline

Use `AKSSEC-Discovery.ps1` for the pre-deploy picture, `AKSSEC-Validate.ps1` for the five baseline controls, and `AKSSEC-Report.ps1` for posture and drift. Objects are referenced by their `<Purpose> – <Scope> – <Stage>` names.

A. `terraform apply` / IaC failures

Symptom	Cause	Remediation
Apply fails on role assignment (`AuthorizationFailed`)	Operating account lacks rights to *write role assignments*	Grant **Owner** or **Contributor + Role Based Access Control Administrator** at the scope; re-run `terraform plan` then `apply`
Subnet / CNI address errors	Subnet too small for Azure CNI (IPs = maxPods × maxNodes + nodes)	Resize the subnet or lower `max_pods`/node count; re-plan. Confirm capacity with `AKSSEC-Discovery.ps1`
Provider / feature not found	`Microsoft.ContainerService` / `Microsoft.PolicyInsights` not registered, or a preview feature not enabled in the region	`az provider register --namespace Microsoft.ContainerService` (and PolicyInsights); *verify the feature/add-on is available in your region*
Version error on `kubernetes_version` / provider	Pinned version no longer offered	Set a currently supported AKS version and provider version; re-plan

B. `kubectl` can't reach the cluster (private API)

Symptom	Cause	Remediation
`kubectl get nodes` times out	You are **not on the cluster's network** — the API is private by design	Run from a workstation with a network path to the private endpoint (VNet peering, VPN, or a Bastion/jumpbox that resolves the private DNS zone)
`Unable to connect ... dial tcp` on the private FQDN	Private DNS zone not resolvable from your location	Ensure the private DNS zone is linked to your management VNet, or use the jumpbox inside the cluster VNet
Auth prompt loops / `error: You must be logged in`	Entra-integrated cluster needs kubelogin	`kubelogin convert-kubeconfig -l azurecli` after `az aks get-credentials`; sign in as a member of `AKS Admins – Baseline – Prod`

C. Access denied for the right people

Symptom	Cause	Remediation
A developer can't act in `app-baseline`	Not a member of `AKS Developers – Baseline – Prod`, or the `RoleBinding` uses the wrong object ID	Confirm group membership; verify the `RoleBinding` subject is the **group object ID**; re-apply `AKSSEC-baseline.yaml`
Everyone is blocked, even admins	Local accounts disabled and Azure RBAC role not assigned in Azure	Assign the Entra admin group the **Azure Kubernetes Service RBAC Cluster Admin** role at the cluster scope; confirm with `AKSSEC-Validate.ps1`
Developer has cluster-wide power	Developer group bound to `cluster-admin` instead of namespaced `edit`	Fix the manifest — developers get a namespaced `RoleBinding` to `edit`, not a `ClusterRoleBinding`

D. Azure Policy for Kubernetes / Gatekeeper

Symptom	Cause	Remediation
Compliance shows blank	Gatekeeper hasn't synced yet	Allow time (constraints sync on an interval); confirm `kubectl get pods -n gatekeeper-system` are running; re-check **Policy** → **Compliance**
A non-compliant pod deploys anyway	Assignment is still in **audit**	Expected in the audit stage — audit reports, it does not block. Move to `AKS Guardrails – NonProd – Deny` only after the gate
Deny blocks a legitimate workload	Guardrail too broad for one case	Create a scoped, justified, time-boxed **Exemption** for that resource — do not weaken the policy for everyone. If urgent, roll the effect back to **audit**
Gatekeeper pods not present	Azure Policy add-on not enabled	Confirm `azure_policy_enabled = true` in the IaC; `AKSSEC-Validate.ps1` reports the add-on state

E. Workload identity not working (pod can't reach Azure)

- Confirm the **federated credential** subject exactly matches `system:serviceaccount:app-baseline:wi-app-baseline` and the **issuer** is the cluster's OIDC issuer URL.

• Confirm the `ServiceAccount` annotation `azure.workload.identity/client-id` holds the **managed identity client id** (Terraform output `workload_identity_client_id`) and the pod carries the label `azure.workload.identity/use: "true"`.

• Confirm the managed identity has **least-privilege access** to the target resource (e.g. Key Vault Secrets User) — a token that can't read the resource looks like an auth failure.

• *Verify workload-identity GA status and the correct annotation keys for your cluster version.*

F. Defender for Containers has no findings

• Confirm the **Containers** plan is enabled on the subscription (**Environment settings**) and the Defender agent/extension deployed to the cluster.

• Image-scan and node-hardening results appear on Defender's own cadence — allow time after enabling; re-run AKSSEC`-Report.ps1` to capture coverage and top findings.

G. Diagnostics quick reference

```
Get-AzContext | Select-Object Subscription, Tenant
Get-AzAksCluster -ResourceGroupName '<rg>' -Name '<cluster>' | Select-Object Name, `
    @{n='PrivateAPI';e={$_.ApiServerAccessProfile.EnablePrivateCluster}}, DisableLocalAccounts
az provider show -n Microsoft.PolicyInsights --query registrationState -o tsv
```

```
kubectl get pods -n gatekeeper-system
kubectl get clusterrolebinding,rolebinding -A | grep baseline
kubectl describe sa wi-app-baseline -n app-baseline
```

H. When to escalate

If the cluster reports the baseline controls (private API, Azure RBAC, policy add-on, OIDC, workload identity) via AKSSEC`-Validate.ps1`, Gatekeeper pods are healthy, the federated credential matches, and either policy compliance or Defender findings still do not populate after the normal evaluation window, collect AKSSEC`-Report.ps1` output plus the cluster and assignment details and open a Microsoft support case.

7. Best Practices — Azure Kubernetes Service (AKS) Secure Baseline

• **Plan before every apply.** Run `terraform plan` (or `az deployment group what-if`) and read the resource list before you apply — on the non-prod cluster first, then production. A blind apply is how a networking change quietly takes the cluster offline.

• **Keep the API private.** Prefer a fully private cluster; if a public endpoint is genuinely unavoidable, lock it to **authorized IP ranges**. Plan the management path (peering / VPN / Bastion jumpbox) *before* you deploy, not after `kubectl` times out.

• **Bind groups, never people, and never cluster-admin by default.** Map **Entra security groups** to least-privilege Kubernetes roles; give developers **namespaced edit**, not cluster-admin. Disable local accounts so there is no shared `cluster-admin` kubeconfig to leak.

• **Audit before you deny, always.** Assign every Azure Policy for Kubernetes guardrail in **audit** first, understand the violations, exempt the legitimate exceptions, then flip the **same** assignment to **deny** on an approved gate. This prevents blocking a deployment real workloads depend on.

• **Name everything and reuse the name.** Use `<Purpose> – <Scope> – <Stage>` for the cluster, resource group, groups, identity, service account, and policy assignments, and reuse it verbatim across IaC, manifests, scripts, and rollback so the estate stays auditable.

• **Go secretless with workload identity.** Federate a **user-assigned managed identity** to a Kubernetes service account and grant it the **narrowest** Azure access it needs (e.g. one Key Vault, secrets-user only). No connection strings, no stored keys, nothing to leak through logs.

• **Turn on Defender for Containers and work the findings.** Enable the plan, act on **image-scan** and **node-hardening** recommendations, and track the secure score each cycle — a rising score is the evidence the baseline is holding.

• **Deny stops new, not running.** A deny policy blocks non-compliant workloads at admission; it does not fix what is already running. Remediate existing workloads separately.

• **Keep a rollback for every change.** `terraform destroy` of the non-prod stack, or setting a policy assignment back to **audit**, is the safety net — write it down before you make the change.

• **Non-prod, then production.** Prove the whole baseline on AKS `Cluster – Baseline – NonProd` before creating AKS `Cluster – Baseline – Prod` from the same code — the same deploy-once, repeat pattern that makes this repeatable.

Where to go next

• **Azure Security Baseline & Governance** — put management-group-level Azure Policy guardrails and RBAC least-privilege around the whole subscription the cluster lives in.

• **Power Platform Secrets in Azure Key Vault** — the secretless pattern extended to the Key Vault the workload identity reads from.

• **Azure Virtual Desktop / Windows 365** — a governed, network-isolated management jumpbox for reaching the private API endpoint.

• Adjacent Azure Solutions in the catalog cover monitoring, cost governance, and network hardening that complement this baseline.

8. Deliverables Summary — Azure Kubernetes Service (AKS) Secure Baseline

Documentation

Document	Purpose
Executive Summary	Business case and what's in the box

Document	Purpose
1. Solution Overview	What the AKS secure baseline is — private networking, Entra + Kubernetes RBAC, Azure Policy (Gatekeeper), workload identity, Defender for Containers; benefits, limitations
2. Requirements Guide	Every prerequisite with Verify / Implement-if-missing
Global Requirements (Azure Solutions)	Category-wide prerequisites, published once
3. Implementation Guide	Exact, ordered steps — plan the IaC, deploy the private cluster, apply RBAC + policy (audit→deny), wire workload identity, enable Defender
5. Testing Guide	Validation procedures with pass/fail gates
6. Troubleshooting Guide	Field issues and remediation
7. Best Practices	Field-proven recommendations and where to go next

Infrastructure-as-Code & scripts

File	What it does
`AKSSEC-Discovery.ps1`	Read-only Az PowerShell discovery — subscription, VNet/subnet capacity, existing clusters, and Defender for Containers plan; a readiness picture before you deploy
`AKSSEC-main.tf`	Terraform for the private AKS cluster — private API endpoint, Azure CNI, Entra + Azure RBAC for Kubernetes, Azure Policy add-on, OIDC issuer + workload identity, and the user-assigned managed identity + federated credential. **Deploy plan-first** (`terraform plan` before `apply`)
`AKSSEC-baseline.yaml`	kubectl manifests — the `app-baseline` namespace, least-privilege RBAC bindings mapping Entra groups to roles, and the workload-identity service account for secretless pod-to-Azure auth
`AKSSEC-Validate.ps1`	Confirms private API, Azure RBAC, the policy add-on/Gatekeeper, OIDC issuer, and workload identity are all in place
`AKSSEC-Report.ps1`	Baseline posture report — private-API state, RBAC, policy assignments and compliance, workload-identity/OIDC, and Defender coverage with top findings

What the buyer receives

- This implementation guide as a branded **Word** document and **PDF**.
- The **Terraform** configuration, **YAML** manifests, and three **Az PowerShell** scripts in a single ZIP — delivered as a **Complete Solution Package (PDF + Word + scripts)**.
- A repeatable process following the repeatable model: **Assess** → **Configure** → **Pilot** → **Validate** → **Expand** — with a plan/what-if gate before every apply and a documented rollback.

Outcome

Kubernetes running securely on Azure — a private, policy-guardrailed cluster with Entra least-privilege identity, secretless workload identity, and Defender for Containers — proven on a non-production cluster before it carries production workloads.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

AKSSEC-Discovery.ps1

Purpose: Read-only discovery for the "Azure Kubernetes Service (AKS) Secure Baseline" solution. Captures the pre-change picture BEFORE any Terraform/Bicep apply: the subscription in context, the target virtual network / subnet and its free-address capacity (Azure CNI needs one IP per pod plus nodes), any existing AKS clusters and whether their API is private, and the Microsoft Defender for Containers plan tier. Makes no changes. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\AKSSEC-Discovery.ps1 powershell -ExecutionPolicy Bypass -File .\AKSSEC-Discovery.ps1 -SubscriptionId '<sub id>' ` -VnetResourceGroup '<rg>' -VnetName '<vnet>' -SubnetName '<subnet>' Requires Az modules (Az.Accounts, Az.Aks, Az.Network, Az.Security). Run Connect-AzAccount first, or the script prompts. EXPECTED RESULTS A single readiness snapshot — subscription, subnet free-IP capacity, existing clusters with private-API state, and Defender for Containers plan status — so you know the estate is ready before you plan the IaC. .NOTES White-label · Azure Solutions. Read-only. The repeatable way. Reports configuration/identifiers only; makes no changes to any resource. > [CmdletBinding()] param([string]$SubscriptionId, [string]$VnetResourceGroup, [string]$VnetName, [string]$SubnetName)

How to run: powershell -ExecutionPolicy Bypass -File .\AKSSEC-Discovery.ps1 powershell -ExecutionPolicy Bypass -File .\AKSSEC-Discovery.ps1 -SubscriptionId '<sub id>' ` -VnetResourceGroup '<rg>' -VnetName '<vnet>' -SubnetName '<subnet>' Requires Az modules (Az.Accounts, Az.Aks, Az.Network, Az.Security). Run Connect-AzAccount first, or the script prompts. EXPECTED RESULTS A single readiness snapshot — subscription, subnet free-IP capacity, existing clusters with private-API state, and Defender for Containers plan status — so you know the estate is ready before you plan the IaC. .NOTES White-label · Azure Solutions. Read-only. The repeatable way. Reports configuration/identifiers only; makes no changes to any resource. > [CmdletBinding()] param([string]$SubscriptionId, [string]$VnetResourceGroup, [string]$VnetName, [string]$SubnetName)

AKSSEC-Report.ps1

Purpose: Read-only reporting for the "Azure Kubernetes Service (AKS) Secure Baseline" solution. Produces one posture snapshot for a named cluster: the private-API / Azure RBAC / policy add-on / OIDC / workload-identity flags, the Azure Policy assignments in the cluster's resource group with their compliance summary, and the Defender for Containers plan tier. Optionally writes the summary to CSV. Makes no changes. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\AKSSEC-Report.ps1 ` -ResourceGroup '<rg>' -ClusterName '<cluster>' Add -CsvPath '.\akssec-posture.csv' to export. Requires Az.Accounts, Az.Aks, Az.PolicyInsights, Az.Security. Connect-AzAccount first. EXPECTED RESULTS A single baseline report — hardening flags, policy assignments + compliant/non-compliant counts, workload-identity state, and Defender coverage — captured at each stage (audit, post-deny, each expansion) so you can show drift over time. .NOTES White-label · Azure Solutions. Read-only. The repeatable way. Reports configuration/compliance only; makes no changes. > [CmdletBinding()] param([Parameter(Mandatory)][string]$ResourceGroup, [Parameter(Mandatory)][string]$ClusterName, [string]$CsvPath)

How to run: powershell -ExecutionPolicy Bypass -File .\AKSSEC-Report.ps1 ` -ResourceGroup '<rg>' -ClusterName '<cluster>' Add -CsvPath '.\akssec-posture.csv' to export. Requires Az.Accounts, Az.Aks,

Az.PolicyInsights, Az.Security. Connect-AzAccount first. EXPECTED RESULTS A single baseline report — hardening flags, policy assignments + compliant/non-compliant counts, workload-identity state, and Defender coverage — captured at each stage (audit, post-deny, each expansion) so you can show drift over time. .NOTES White-label · Azure Solutions. Read-only. The repeatable way. Reports configuration/compliance only; makes no changes. > [CmdletBinding()] param([Parameter(Mandatory)][string]$ResourceGroup, [Parameter(Mandatory)][string]$ClusterName, [string]$CsvPath)

AKSSEC-Validate.ps1

Purpose: Read-only validation for the "Azure Kubernetes Service (AKS) Secure Baseline" solution. After the IaC apply (non-prod first), checks the five baseline controls on a named cluster and prints a PASS/REVIEW line for each: private API endpoint, Microsoft Entra + Azure RBAC for Kubernetes with local accounts disabled, the Azure Policy add-on (Gatekeeper), the OIDC issuer, and workload identity. Optionally checks operator tooling. Makes no changes. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\AKSSEC-Validate.ps1 ` -ResourceGroup '<rg>' -ClusterName '<cluster>' Add -CheckTools to verify terraform / kubectl / kubelogin are installed locally. Requires Az.Accounts, Az.Aks. Run Connect-AzAccount first, or the script prompts. EXPECTED RESULTS A per-control PASS/REVIEW summary. All PASS means the cluster is at the secure baseline and ready for the kubectl RBAC/policy manifests and Defender for Containers. .NOTES White-label · Azure Solutions. Read-only. The repeatable way. Reports state only; makes no changes to the cluster. > [CmdletBinding()] param([Parameter(Mandatory)][string]$ResourceGroup, [Parameter(Mandatory)][string]$ClusterName, [switch]$CheckTools)

How to run: powershell -ExecutionPolicy Bypass -File .\AKSSEC-Validate.ps1 ` -ResourceGroup '<rg>' -ClusterName '<cluster>' Add -CheckTools to verify terraform / kubectl / kubelogin are installed locally. Requires Az.Accounts, Az.Aks. Run Connect-AzAccount first, or the script prompts. EXPECTED RESULTS A per-control PASS/REVIEW summary. All PASS means the cluster is at the secure baseline and ready for the kubectl RBAC/policy manifests and Defender for Containers. .NOTES White-label · Azure Solutions. Read-only. The repeatable way. Reports state only; makes no changes to the cluster. > [CmdletBinding()] param([Parameter(Mandatory)][string]$ResourceGroup, [Parameter(Mandatory)][string]$ClusterName, [switch]$CheckTools)

AKSSEC-baseline.yaml

AKSSEC-main.tf

Purpose: Deploy a private, hardened Azure Kubernetes Service cluster as code: - Private API endpoint (no public IP), Azure CNI inside an existing subnet - Microsoft Entra ID integration + Azure RBAC for Kubernetes (local accounts disabled) - Azure Policy add-on (Gatekeeper) for admission guardrails - OIDC issuer + workload identity, plus a user-assigned managed identity and a federated credential for secretless pod-to-Azure auth HOW TO RUN (ALWAYS plan first — never blind-apply; non-prod cluster first) az login terraform init terraform plan -out aksbaseline.plan # <-- review the resource list (the gate) terraform apply aksbaseline.plan # only after review + approval Rollback: terraform destroy (destroys the NON-PROD stack this file created) EXPECTED RESULTS A private AKS cluster with Entra+Azure RBAC, the policy add-on, OIDC/workload identity, and a federated managed identity — ready for the kubectl RBAC/policy manifests (AKSSEC-baseline.yaml) and Defender for Containers. NOTES White-label. Plan/what-if first. Non-prod first. Feature/add-on/API versions marked "verify" move between GA and preview — verify current availability in your region before relying on them. Placeholder values (contoso, subnet IDs, group

IDs) MUST be set.